Megamind

Egocentric Memory for Meta Quest 3: Design Document

Project lead: Devin · Team: 4 engineers

v0.1 — Hackathon build

Abstract

Megamind gives a Meta Quest 3 wearer an AI assistant that remembers what they have seen. The headset's passthrough cameras are sampled continuously; keyframes are filtered, described, embedded, and stored as *memory tokens* in a vector database. A Claude-based agent exposes a `recall_memory` tool; when invoked, relevant tokens are retrieved and handed to a Qwen vision-language "reader" sub-agent that attends over the concatenated tokens and returns grounded evidence. Claude composes the final answer, which is delivered through a voice-first AR chat interface on the headset. This document fixes the architecture, interfaces, model choices, work split and risk plan so that four engineers and several coding agents can build it in parallel.

Contents

1 Goals and Scope **3**
- 1.1 Goal . . . 3
- 1.2 In scope . . . 3
- 1.3 Explicitly out of scope . . . 3

2 Key Design Decisions **3**

3 System Architecture **4**
- 3.1 Components . . . 4

4 Stream 1 — Vision Capture and Memory Tokenization **5**
- 4.1 Capture on Quest 3 . . . 5
- 4.2 Filtering . . . 5
- 4.3 Memory token . . . 5

5 Stream 2 — Memory Retrieval and the Reader Sub-Agent **6**
- 5.1 Tool interface exposed to Claude . . . 6
- 5.2 Retrieval . . . 6
- 5.3 Reader sub-agent (Qwen-VL) . . . 6
- 5.4 Latency budget (target $\leq$ 8 s to first spoken word) . . . 7

6 Stream 3 — AR Chat Interface **7**

7 API Contract **7**

8 Repository Layout **7**

9 Work Split and Coordination **8**

10 Risks and Mitigations **9**

11 Configuration **9**

12 Open Questions **9**

1 Goals and Scope

1.1 Goal

Demonstrate, live, a wearer asking the headset questions whose answers depend on things they saw earlier—"Where did I leave my keys?", "What was the name on that whiteboard?", "Who did I talk to after lunch?"—and receiving correct, grounded answers with a visual citation of the remembered moment.

1.2 In scope

- Continuous passthrough capture on Quest 3, sampled and uploaded as keyframes.
- Server-side filtering, captioning/OCR, embedding and storage of memory tokens.
- A Claude agent with a `recall_memory` tool; a Qwen-VL reader sub-agent that performs attention over retrieved tokens.
- Voice-first AR chat panel on the headset (push-to-talk in, text + speech out), with the cited memory thumbnail shown.
- A laptop/webcam feeder that emulates the headset so the backend is testable without hardware.

1.3 Explicitly out of scope

- Training or fine-tuning any model; learned "soft" memory tokens.
- On-device inference. Everything heavy runs on the server.
- Multi-user memory, auth, or long-term privacy controls beyond a purge button.
- Quest 2 / Quest Pro support (no camera API).

2 Key Design Decisions

Decision

D1 — One Unity app on the headset does both capture and chat. Horizon OS keeps a single immersive app in the foreground and suspends others, so a separate chat app or WebXR page would stop capture. Streams 1 and 3 therefore share one Android APK.

Decision

D2 — A memory token is a record, not a learned vector. Each token = one keyframe at time t with: a dense caption, OCR text, entity list, a text embedding, an image embedding, and the JPEG thumbnail. "Attention over tokens" is performed by the Qwen-VL reader, which receives the concatenated captions *and* thumbnails as real vision/text tokens in its context. This is honest to the pitch and buildable in a day.

Decision

D3 — Two-model agent: Claude orchestrates, Qwen-VL reads memory. Claude (Sonnet) is the user-facing agent and tool caller. The `recall_memory` tool retrieves top-k tokens and invokes a Qwen2.5-VL (or Qwen3-VL) reader that answers the sub-question over those tokens with citations. Claude never sees raw frames unless the reader's evidence is insufficient, keeping its context small and latency bounded.

Decision

D4 — All intelligence is server-side; the headset is a thin client. The headset uploads JPEG keyframes and audio, and renders text/audio/thumbnails. Every server endpoint is exercisable with `curl`, so backend progress never waits on a headset build.

Decision

D5 — LanceDB as the memory store. Embedded, pip-installable, supports vector + full-text + metadata filtering. No Docker, no infra. Wrapped behind a `MemoryStore` interface so it can be swapped for pgvector/Qdrant if needed.

Decision

D6 — Voice first. Push-to-talk (controller button or hand pinch) → server STT. Replies are spoken (server TTS) and shown on a world-locked panel. Typing in VR is not supported.

3 System Architecture

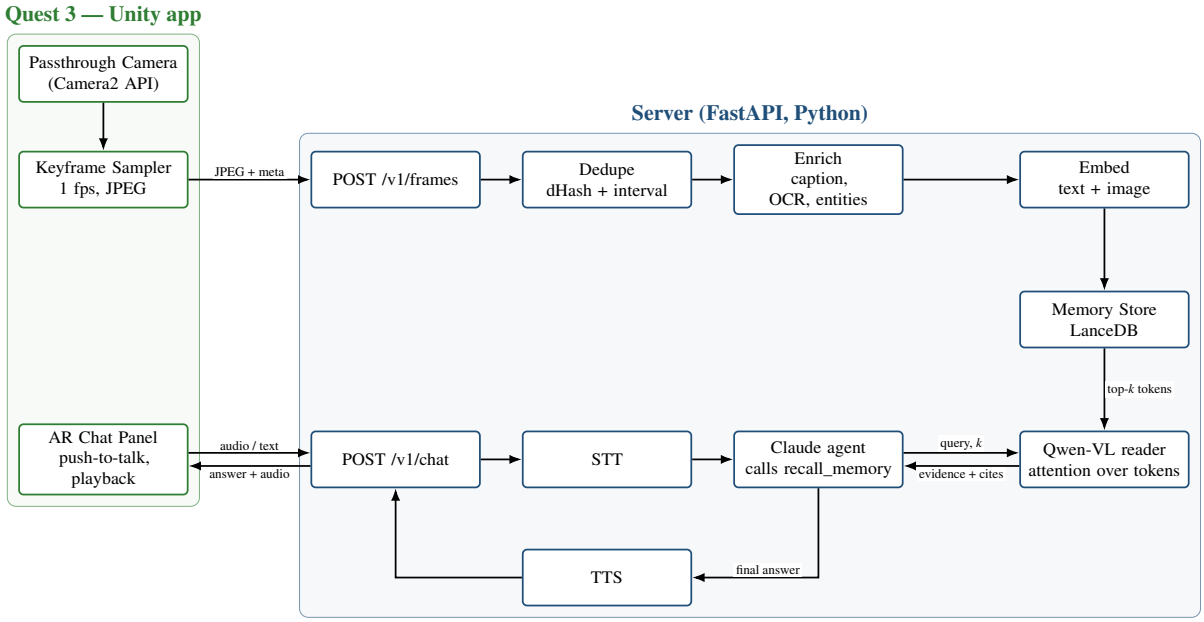


Figure 1: End-to-end data flow. Green: headset. Blue: server. Top row is Stream 1 (ingest); bottom rows are Streams 2–3 (agent and chat).

3.1 Components

Headset app (Unity 2022.3 LTS / Unity 6, Meta XR SDK $\geq$ v74).
Reads the passthrough camera via Meta's Passthrough Camera API (exposed as Android Camera2; the Meta sample wraps it in `WebCamTextureManager`). Samples one frame per second, encodes JPEG ($\leq$ 960 px wide, quality 75), and POSTs to `/v1/frames` with device id, UTC timestamp and head pose. Renders a world-locked chat panel; records microphone on push-to-talk; plays returned audio.

Ingest service (`server/megamind/ingest`).
Accepts frames, drops near-duplicates (perceptual dHash Hamming distance < 6 within the last 30 s, and minimum 2 s gap), then enqueues to a worker that produces a memory token (Section 4.3) and writes it to the store.

Memory store (`server/megamind/store`).
LanceDB table of memory tokens with two vector columns (text, image) and full-text search on `caption` + `ocr_text`. Hybrid query: vector + BM25 fused by reciprocal rank, pre-filtered by time range.

Agent service (`server/megamind/agent`)**.**
STT → Claude with tools → (`recall_memory` → store → Qwen-VL reader) → Claude answer → TTS. Returns text, audio, and the ids/thumbnails of memories cited.

Feeder (`tools/feeder.py`)**.**
Emulates the headset from a laptop webcam or a video file. Used for all backend development and as a demo fallback.

4 Stream 1 — Vision Capture and Memory Tokenization

4.1 Capture on Quest 3

- Requires Quest 3 / 3S on Horizon OS v74+ with developer mode. Android manifest needs `horizonos.permission.HEADSET_CAMERA` and `android.permission.CAMERA`; the user grants it at runtime.
- Use the `PassthroughCameraApiSamples` pattern: `WebCamTextureManager` selects the left RGB camera, max 1280×960 @ 30 fps. We read `WebCamTexture` into a `Texture2D` once per second, downscale to 960 px wide and `EncodeToJPG(75)`.
- Upload with `UnityWebRequest` multipart POST, fire-and-forget with a bounded queue (drop oldest if > 10 pending). Target ∼80–120 KB/frame, ∼1 Mbit/s.
- Also send head pose (position + yaw) from `OVRCameraRig`; it is cheap and enables "where" answers later.
- Do not attempt video streaming or on-device inference.

4.2 Filtering

Two-stage, deliberately simple:

1. **Client:** fixed 1 fps sample.
2. **Server:** dHash (8×8) against the last 30 s of accepted frames; reject if Hamming distance $<$ 6. Enforce a 2 s minimum gap. Reject frames with mean luminance < 20 (lens covered/dark). Expected keep rate 10–30%.

4.3 Memory token

```
MemoryToken {
  id:             str   # ulid
  device_id:      str
  ts:             float # unix seconds, UTC
  caption:        str   # 1-3 dense sentences from the VLM
  ocr_text:       str   # any legible text, verbatim
  entities:       list[str]   # objects, people descriptors, places
  pose:           {x,y,z,yaw} | null
  text_vec:       float[1024] # embedding of caption + ocr + entities
  image_vec:      float[768]  # SigLIP image embedding
  thumb_path:     str   # JPEG on disk, served at /v1/thumbs/{id}.jpg
  frame_hash:     str   # dHash hex
  episode_id:     str | null  # filled by consolidation (stretch)
}
```

Enrichment model. Qwen2.5-VL-7B-Instruct (or Qwen3-VL) via an OpenAI-compatible endpoint (vLLM locally if a GPU is available, otherwise a hosted provider). One call per keyframe with a fixed JSON-output prompt producing `caption`, `ocr_text`, `entities`. Fallback: Claude Haiku with vision. Prompt emphasises *retrievable* detail: object locations relative to surfaces, legible text verbatim, people descriptors (no identity claims), room type.

Embeddings. Text: a hosted embedding model (Voyage `voyage-3` or OpenAI `text-embedding-3-small`) or local `bge-m3`. Image: SigLIP-base (`open_clip`) for visual-similarity queries. Both configurable via environment variables.

Consolidation (stretch). Every 10 minutes, summarise accepted tokens into an *episode* token ("10:05–10:15: in the kitchen, made coffee, phone on counter") to help vague queries. Optional.

Risk

Camera API bring-up is the single largest risk: SDK/OS version mismatch, permission not granted, `WebCamTexture` returning black. Mitigation: hour-0 spike using Meta's sample scene unchanged before any custom code; feeder as fallback for every other stream.

5 Stream 2 — Memory Retrieval and the Reader Sub-Agent

5.1 Tool interface exposed to Claude

```
recall_memory(
  query: str,              # natural-language description of what to find
  time_from: str | null,   # ISO-8601; null = no lower bound
  time_to:   str | null,
  k: int = 12,             # tokens to retrieve
  mode: "semantic" | "visual" | "hybrid" = "hybrid"
) -> {
  evidence:    str,        # reader's grounded answer to `query`
  confidence:  "high" | "medium" | "low" | "not_found",
  cited:       [{id, ts, caption, thumb_url}],
  all_retrieved: [{id, ts, caption}]
}
```

Claude's system prompt instructs it to call `recall_memory` whenever the user's question references past perception, to translate relative times ("this morning", "ten minutes ago") into absolute ISO ranges using the injected current time, and to answer "I don't have a memory of that" when confidence is `not_found` rather than guess.

5.2 Retrieval

1. Pre-filter by `ts` range (if any) and device.
2. Hybrid search: cosine on `text_vec` (query embedding) + BM25 on caption/OCR, fused with RRF. In `visual` mode, embed the query with SigLIP's text tower and search `image_vec`. `hybrid` unions both.
3. Recency tie-break: among near-equal scores prefer newer.
4. Return top-k (default 12; hard cap 24 to bound reader cost).

5.3 Reader sub-agent (Qwen-VL)

The reader receives, in one prompt, the concatenated tokens: for each, `[id] [timestamp] caption / ocr` plus the thumbnail image, followed by the query. It is asked to answer only from the tokens, cite token ids, and emit a confidence label as JSON. This is the component that literally performs attention across memories. Qwen2.5-VL-7B handles 12–24 images comfortably; use 72B via a hosted provider if quality is lacking.

Why two models. Claude keeps the conversation, tool-use discipline and final phrasing; the reader does cheap, parallelisable, image-heavy evidence extraction. Claude's context never fills with thumbnails, so multi-turn chat remains fast.

5.4 Latency budget (target $\leq$ 8 s to first spoken word)

STT (Whisper / Deepgram)	0.8 s
Claude turn 1 (decide tool call)	1.5 s
Retrieval (LanceDB)	0.1 s
Reader (Qwen-VL, 12 images)	2.5 s
Claude turn 2 (compose answer)	1.5 s
TTS (first chunk)	0.8 s
Total	$\approx$ 7.2 s

Stream Claude's final text to the panel while TTS renders so the UI feels responsive.

6 Stream 3 — AR Chat Interface

- **Scene:** `OVRCameraRig` with Passthrough enabled (Building Block), a world-locked `Canvas` panel 0.6 m wide placed 1 m ahead on first use and re-anchorable by a controller gesture.
- **Input:** push-to-talk on controller trigger (hand-tracking pinch as a stretch). Unity `Microphone` $\rightarrow$ 16 kHz mono WAV $\rightarrow$ `POST /v1/chat` (multipart). A "listening" indicator shows on the panel.
- **Output:** transcript bubbles (user/assistant), cited memory thumbnails rendered beside the assistant bubble (fetched from `thumb_url`), audio played via `AudioSource` from the returned WAV URL.
- **Status HUD:** small indicator of capture state (frames/min accepted) and server connectivity.
- **Networking:** server reached over Tailscale (plain HTTP on the tailnet, `usesCleartextTraffic=true`) or an HTTPS ngrok tunnel. Base URL in a `ScriptableObject` config, overridable via a settings panel.

7 API Contract

All endpoints are JSON unless noted. Base path `/v1`. Authoritative version: `docs/api.md` in the repo.

Endpoint	Description
`GET /health`	`{status, store_count, models}`
`POST /frames`	multipart: `image` (JPEG), `meta` JSON `{device_id, ts, pose?}`. Returns `202 {frame_id, accepted, reason}`. Non-blocking; enrichment is async.
`GET /memories`	Debug search. Query params `q, from, to, k, mode`. Returns tokens without vectors.
`GET /thumbs/{id}.jpg`	Keyframe thumbnail.
`POST /chat`	multipart or JSON: `session_id`, and either `text` or `audio` (WAV). Returns `{transcript, answer, audio_url, cited:[{id,ts,caption,thumb_url}], tool_calls}`.
`GET /audio/{id}.wav`	TTS output.
`DELETE /memories`	Purge all memory for a device (privacy / demo reset).

8 Repository Layout

```
megamind/
  README.md
  docs/
    megamind-design.tex / .pdf     # this document
    api.md                         # authoritative API contract
  server/                          # Python 3.11, FastAPI, uv/pip
    pyproject.toml
    megamind/
      main.py                      # app factory, routers
      config.py                    # env-driven settings
      models.py                    # pydantic schemas (MemoryToken, ...)
      ingest/   {router.py, filter.py, enrich.py, worker.py}
      store/    {base.py, lancedb_store.py}
      agent/    {router.py, claude_agent.py, reader.py, stt.py, tts.py, tools.py}
    tests/
  tools/
    feeder.py                      # webcam/video -> /v1/frames
    ask.py                         # CLI chat client
  headset/
    README.md                      # Unity project setup steps
    Megamind/                      # Unity project (committed sans Library/)
      Assets/Megamind/Scripts/{FrameUploader.cs, ChatClient.cs, ChatPanel.cs, ...}
```

9 Work Split and Coordination

Owner	Scope	Agent support
Engineer 1 — Headset capture	Unity project, Passthrough Camera bring-up, `FrameUploader.cs`, manifest/permissions, APK builds	Devin writes C#, cannot run on device
Engineer 2 — Headset UI/voice	Chat panel, push-to-talk, audio playback, thumbnails, config	Devin writes C#
Engineer 3 — Ingest + store	Reviews/tests agent-built ingest service with feeder; tunes filter and caption prompt	Devin agent builds end-to-end
Engineer 4 — Agent + demo	Reviews/tests agent-built agent service; writes demo script; records fallback video	Devin agent builds end-to-end
Devin (lead)	API contract, repo skeleton, integration, agent coordination, unblocking	—

Integration rule. Headset and server only communicate through `docs/api.md`. Any change to the contract is a PR that updates `api.md`, the pydantic models and the C# DTOs together.

Phased plan.

1. **Hours 0–3: de-risk.** Camera sample runs on headset (E1). Repo skeleton + `/health` + feeder merged (Devin). Model API keys verified.
2. **Hours 3–12: vertical slice.** Feeder → ingest → store → `GET /memories` works. `POST /chat` with text answers a question about feeder frames. Headset uploads real frames.
3. **Hours 12–24: full loop.** Voice in/out on headset, thumbnails on panel, reader sub-agent live. First end-to-end "where are my keys" on hardware.
4. **Final hours: demo hardening.** Fixed demo script, prompt tuning, latency trims, purge/reset, fallback video.

10 Risks and Mitigations

Risk	Sev.	Mitigation
Passthrough Camera API fails to deliver frames	High	Hour-0 spike with Meta sample; feeder fallback; phone camera as backup capture device
Unity build loop too slow	High	Thin client; all logic server-side and curl-testable; one build engineer only
Reader/VLM provider latency or outages	Med	Pluggable OpenAI-compatible endpoint; Claude Haiku vision fallback
Poor recall on demo queries	Med	Script the demo; tune caption prompt for object placement + OCR; hybrid search with time filter
Network from headset to laptop	Med	Tailscale on both; ngrok as backup; base URL editable at runtime
Scope creep ("real" memory tokens, consolidation)	Med	Stretch goals gated behind a working demo
Privacy of bystanders	Low (hackathon)	Visible capture indicator, local storage, purge endpoint, no identity recognition in prompts

11 Configuration

All secrets and model selections via environment variables (`server/.env.example`):

```
ANTHROPIC_API_KEY=...
CLAUDE_MODEL=claude-sonnet-4-5
READER_BASE_URL=http://localhost:8001/v1      # vLLM or hosted OpenAI-compatible
READER_API_KEY=...
READER_MODEL=Qwen/Qwen2.5-VL-7B-Instruct
ENRICH_BASE_URL=${READER_BASE_URL}            # same model by default
EMBED_PROVIDER=openai|voyage|local
EMBED_MODEL=text-embedding-3-small
STT_PROVIDER=openai|deepgram
TTS_PROVIDER=openai|elevenlabs
LANCEDB_PATH=./data/lancedb
THUMB_DIR=./data/thumbs
```

12 Open Questions

- GPU availability for local Qwen-VL via vLLM, or hosted only?
- Hackathon end time (fixes the phase boundaries above).
- Preferred STT/TTS vendor given available credits.